# Keeping Engineers in the Loop: Explainable Requirement Defect Screening Across Software Requirements Specifications

Awaiz Ahmed
*Al Ain University*
*United Arab Emirates*
202310183@aau.ac.ae

Mohammed Umar
*Al Ain University*
*United Arab Emirates*
202240077@aau.ac.ae

**Abstract—Requirements written in natural language frequently contain ambiguity, incompleteness, and statements that are difficult to validate consistently. This experience report presents a human-centered requirement-validation workflow built around a heterogeneous source corpus of 79 software requirements specification documents. The workflow extracted 9,165 requirement-like statements from this corpus and curated a manually reviewed, single-reviewer adjudicated gold set of 110 requirements, which is the basis for all reported evaluation results. Three screening conditions were compared on this gold set: the original seed heuristic, a flat rule-based screen, and an explainable precedence-based screen that returns a defect label, rationale, source span, rewrite guidance, and confidence. The seed heuristic achieved 0.573 accuracy and 0.499 macro-F1, the flat rule screen achieved 0.500 accuracy and 0.478 macro-F1, and the explainable screen achieved 0.855 accuracy and 0.891 macro-F1. The main finding is that explanation-oriented screening substantially improves practical review support in this corpus, while the remaining errors are dominated by formatting artifacts, truncated list boundaries, and contextual judgments that still require human reviewers. The contribution is not autonomous requirement assessment; it is an auditable human-in-the-loop workflow for prioritizing requirement-quality review.**

***Index Terms—requirements engineering, human-centered AI, explainable AI, requirement validation, requirements quality, human-in-the-loop review***

## I. Introduction

Requirements engineering is a human-centered and socio-technical activity. Engineers, analysts, and stakeholders use natural language to negotiate intended system behavior, constraints, interfaces, and acceptance expectations. Because these artifacts guide design, implementation, testing, procurement, and maintenance, defects in requirements can propagate into later lifecycle stages where correction is more expensive [1].

Natural-language requirements are especially difficult to validate at scale. A statement may look syntactically well formed while still being ambiguous, missing an acceptance condition, or difficult to test. Human review is therefore essential, but large legacy specifications make exhaustive inspection slow and inconsistent. This motivates assisted review workflows that can triage likely defects without replacing the reviewer.

This paper reports an experience-oriented empirical study of that problem. The workflow extracts requirement-like statements from heterogeneous files, constructs a manually reviewed subset, and compares three screening conditions. The design emphasizes explainability: the strongest condition does not only return a label, but also a rationale, source span, rewrite guidance, and confidence score. This matches the workshop theme of keeping humans in control as AI and automation enter requirements engineering workflows.

The paper makes three contributions. First, it reports a processed corpus of 79 mixed-format SRS documents from which 9,165 requirement-like statements were extracted. Second, it defines and applies a compact defect taxonomy covering ambiguity, incompleteness, non-testability, and clean requirements. Third, it evaluates a human-in-the-loop explainable screening condition against simpler baselines on a manually reviewed gold set of 110 requirements curated from the extraction corpus.

*Artifact availability:* The reproducibility package, including the curated sample requirements, annotation guide, evaluation script, confusion matrices, and summary results, is available at https://github.com/4waiz/hcare2026_awaiz.

## II. Related Work and Motivation

Requirements quality has long been connected to clarity, completeness, consistency, and verifiability. ISO/IEC/IEEE 29148 positions requirements engineering as a lifecycle process and emphasizes well-formed requirements that support later verification and validation [1]. Glinz shows that quality attributes and their terminology are definitional problems as much as detection problems, which cautions against treating defect labels as self-evident [2]. A systematic mapping of empirical research on requirements quality confirms that most studies target concrete text-level defects and calls for more evidence on how quality tooling actually supports practitioners [3]. Zowghi and Gervasi further show that completeness, consistency, and correctness interact, so repairing one attribute can affect another; this argues for keeping an accountable human decision-maker in the repair loop [4].

Closer to this study, Femmer et al. introduced requirements smells and showed that lightweight defect indicators can help reviewers identify quality problems rapidly, while still requiring human interpretation of each finding [5]. Berry and Kamsties catalogued linguistic sources of ambiguity in specifications and

cautioned that some ambiguity is benign in context, so automated flags need human adjudication rather than automatic rejection [6].

Recent large language model (LLM) and AI-assisted requirements engineering work renews this direction. Surveys and early studies report opportunities in elicitation, analysis, simplification, traceability, and validation, but they also identify challenges around reproducibility, trust, data access, and user-centered deployment [7], [8]. Luitel et al. use language models to assist requirements completeness, illustrating the same assistive framing adopted here: the tool proposes, the engineer decides [9].

The workshop context also demands attention to how AI output is presented to engineers. Guidelines for human-AI interaction recommend making clear why a system acted as it did and supporting efficient correction of mistakes [10], and human-centered AI argues for designs that amplify rather than replace human control [11]. Explanation techniques such as LIME motivate span-level evidence for classifier decisions [12], and Doshi-Velez and Kim frame interpretability as a requirement when decisions carry accountability [13]. Requirements decisions are accountable human decisions, not merely text-classification outputs. A label alone is weak evidence: a reviewer needs to know why a statement was flagged, which phrase triggered the flag, and how the requirement might be rewritten. This motivates the explainable condition evaluated in this paper.

## III. Research Questions

**RQ1.** How accurately can an explainable screening workflow detect ambiguity, incompleteness, non-testability, and clean requirements in a manually reviewed SRS subset?

**RQ2.** How does the explainable screening condition compare with the original seed heuristic and a flat rule-based screen?

**RQ3.** What error patterns remain after explainable screening, and what do they imply for human-in-the-loop requirements review?

## IV. Corpus and Extraction Pipeline

The supplied archive contained 79 SRS-related documents in PDF, DOC, HTML, HTM, and RTF formats. A staged extraction workflow was required because no single parser handled the full archive reliably. PDFs were processed through text extraction, DOC files through document-conversion utilities, and markup-based files through format-specific parsers. The goal was not perfect document reconstruction; it was to build a review-oriented corpus suitable for triage and analysis.

Requirement-like statements were extracted when they contained normative modal language such as shall, must, or should, or when they matched likely actor-action requirement patterns. The resulting dataset stores document identity, requirement identity, extracted text, word count, extraction confidence, weak lexical cues, incompleteness cues, seeded labels, human-reviewed labels, and predicted labels from each screening condition.

From this source corpus of 79 mixed-format SRS documents, the pipeline extracted 9,165 requirement-like statements. This extraction corpus served as the source pool for curation and triage; it was not manually evaluated in full. For the evaluation reported in this paper, we curated and manually reviewed a gold set of 110 requirements, described in Section V. All reported metrics refer to this reviewed gold set.

Table I summarizes the corpus. The median document contributed 52 extracted statements, while the largest document contributed 1,089, confirming that review support must handle both small and large specifications.

TABLE I. Corpus Overview

| Metric | Value |
|---|---|
| Documents processed | 79 |
| Requirement-like statements extracted | 9,165 |
| Manually reviewed gold-set rows | 110 |
| Candidate contradiction pairs | 220 |
| Document formats | PDF, DOC, HTML, HTM, RTF |

## V. Defect Taxonomy and Reviewed Set

The study uses one primary label per requirement statement. Clean means the statement is specific enough to review and does not show a primary defect under the taxonomy. Ambiguity covers vague, subjective, or multiply interpretable wording such as appropriate, adequate, easily, secure, intuitive, or broad uses of support. Incompleteness covers missing conditions, unfinished list introductions, placeholders, truncated sentence endings, and incomplete extracted fragments. Non-testability covers statements that do not express an objectively verifiable system behavior as written.

The taxonomy deliberately concentrates on ambiguity, incompleteness, and non-testability rather than attempting to cover every published quality attribute. These three defect types are among the most common and highest-impact quality problems reported for natural-language requirements [1], [3], [5]: they directly affect how stakeholders interpret a statement, how it is implemented, and whether it can be verified and accepted. They are also well suited to human-centered screening, because deciding whether a flagged statement is genuinely defective still requires engineering judgment about context and intent [6]. The paper does not claim that this taxonomy covers all requirement quality attributes; consistency, traceability, and other attributes remain outside its scope [4].

A 110-row curated subset was single-reviewer adjudicated. The subset was designed to preserve defect diversity rather than mirror the full-corpus frequency distribution, so it is a curated gold set, not a random sample of the 9,165 extracted statements. Table II and Fig. 1 report the reviewed label distribution. The non-testability count is low because several machine-seeded non-testability rows became clean or ambiguous after review; this reveals that simple checks often confuse missing measurement signals with genuinely untestable requirements.

TABLE II. Reviewed Gold-Set Composition

| Label | Rows | Share |
|---|---|---|
| Clean | 40 | 36.4% |
| Ambiguity | 29 | 26.4% |

| | | |
|---|---|---|
| Non-testability | 1 | 0.9% |
| Incompleteness | 40 | 36.4% |

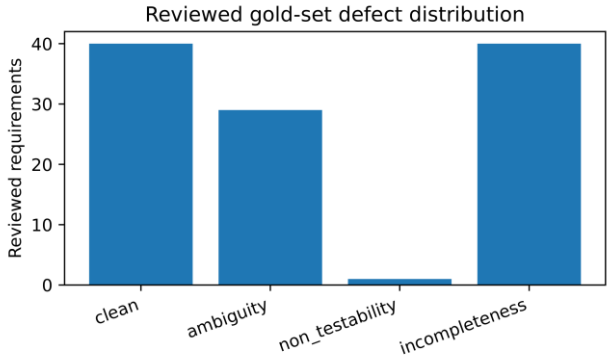


Fig. 1. Reviewed gold-set label distribution after single-reviewer adjudication.

## VI. Screening Conditions

Three conditions were evaluated.

*Seed heuristic.* The first condition uses the original machine-seeded labels produced during corpus construction. It is intentionally simple and depends heavily on weak lexical cues and extraction signals.

*Flat rule screen.* The second condition applies a direct rule set. It prioritizes obvious incompleteness markers such as trailing colons, placeholders, and truncated endings; then flags common vague terms; then uses a conservative non-testability rule; otherwise it predicts clean.

*Explainable precedence-based screen.* The third condition uses a precedence-based classifier with reviewer-facing explanations. It prioritizes extraction and list-boundary problems before ambiguity, because incomplete fragments can otherwise be misclassified as vague requirements. For each statement it returns a primary label, source span, rationale, rewrite guidance, and confidence. Table III shows the output schema.

TABLE III. Explainable Output Schema

| Field | Purpose |
|---|---|
| Defect label | Primary prediction used for evaluation |
| Rationale | Reviewer-facing reason for the decision |
| Source span | Phrase or boundary that triggered the flag |
| Rewrite guidance | Direction for improving the statement |
| Confidence | Triage signal, not a replacement for review |

## VII. Results

Table IV reports the completed comparison on the 110-row reviewed gold set; Fig. 2 summarizes macro-F1 by condition. The seed heuristic achieved 0.573 accuracy and 0.499 macro-F1. The flat rule screen achieved 0.500 accuracy and 0.478 macro-F1. The explainable precedence-based screen achieved the strongest result, with 0.855 accuracy and 0.891 macro-F1.

TABLE IV. Model-Comparison Results on the Reviewed Gold Set

| Condition | Acc. | Prec. | Rec. | F1 |
|---|---|---|---|---|
| Seed heuristic | 0.573 | 0.586 | 0.685 | 0.499 |
| Flat rule screen | 0.500 | 0.654 | 0.621 | 0.478 |
| Explainable screen | 0.855 | 0.911 | 0.891 | 0.891 |

Table V reports per-class results for the explainable condition. The condition performed best for ambiguity and non-testability, while the clean and incompleteness labels show the remaining tension between valid but broad requirements and truncated extraction artifacts. The single non-testability case should not be over-interpreted, but it is useful because it shows why rare defect classes need targeted sampling in future work.

TABLE V. Per-Class Performance of the Explainable Screen

| Label | Support | Prec. | Rec. | F1 |
|---|---|---|---|---|
| Clean | 40 | 0.750 | 0.975 | 0.848 |
| Ambiguity | 29 | 0.893 | 0.862 | 0.877 |
| Non-testability | 1 | 1.000 | 1.000 | 1.000 |
| Incompleteness | 40 | 1.000 | 0.725 | 0.841 |

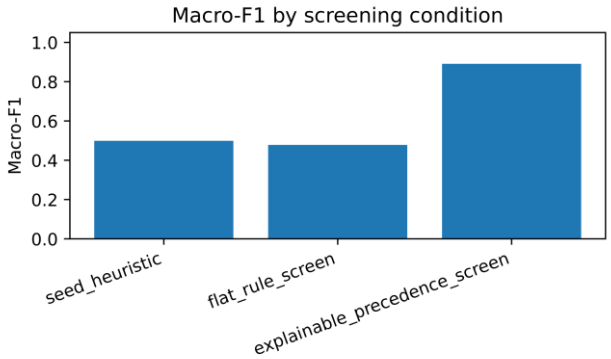


Fig. 2. Macro-F1 by screening condition on the reviewed gold set.

## VIII. Qualitative Error Analysis

The strongest qualitative pattern was that document formatting and requirement quality are entangled. A trailing colon, a placeholder such as *xxx*, or a sentence fragment ending after *to aid in* often indicates an extraction boundary rather than an authoring decision. However, the reviewer still cannot validate the row as written. The explainable condition handled many of these cases by explicitly marking the boundary as the source span.

A second pattern was ambiguous capability language. Terms such as *support*, *adequate*, *appropriate*, *secure*, *intuitive*, and *fast enough* frequently require domain-specific thresholds or externally defined criteria. The source-span output is valuable here because it directs the reviewer to the exact phrase that must be bounded.

A third pattern was over-flagging of testability. Simple rules often classified statements such as "the system shall allow an administrator to create a user" as non-testable because they lacked explicit performance thresholds. Human review treated many such statements as clean functional requirements because the action is objectively observable. This explains why the original seed heuristic produced many false non-testability positives.

Table VI grounds these patterns in twenty representative cases drawn from the reviewed gold set. The cases cover all four labels, mixed-cue cases such as a vague term combined with a truncated list introduction, and an over-flagging case in which

the flat rule screen mislabels a clean administrative requirement as non-testable. Each row pairs a shortened requirement excerpt with the explanation cue surfaced during screening and the reviewer action it suggests. The same twenty cases, with full text, source documents, and the predictions of all three screening conditions, are provided in the repository file representative_20_samples.csv.

TABLE VI. REPRESENTATIVE REQUIREMENT SCREENING EXAMPLES FROM THE REVIEWED GOLD SET (FULL ROWS: REPRESENTATIVE_20_SAMPLES.CSV)

| ID | Requirement excerpt | Gold label | Explanation cue | Suggested reviewer action |
|---|---|---|---|---|
| S01 | The system shall be easily updatable for fixes and patches. | Ambiguity | vague adverb 'easily' | Define measurable update criteria |
| S02 | In case of error it should provide users with appropriate help messages. | Ambiguity | subjective term 'appropriate' | Specify help content per error class |
| S03 | Adequate transmission capacity shall be available to connect the Black Start facility to … | Ambiguity | unquantified 'adequate' | Set a numeric capacity threshold |
| S04 | … of access permissions to administrative features and access permissions must be secure. | Ambiguity | unbounded quality term 'secure' | Reference concrete security controls |
| S05 | The response to a change in the orientation should be fast enough to avoid interrupting the … | Ambiguity | vague bound 'fast enough' | Quantify maximum response latency |
| S06 | The interface must remain consistent among various web browsers and be intuitive to the customer. | Ambiguity | subjective term 'intuitive' | Define usability acceptance criteria |
| S07 | … the Center shall be able to support the following device control command for a HOV Lane: | Incompl. | trailing colon; command list missing | Recover list from source document |
| S08 | The PMS should be easily installable by using: | Incompl. | trailing colon (also vague 'easily') | Complete installation-method list |
| S09 | … to a Client Services email distribution list if the user is directed to the xxx error page. | Incompl. | placeholder 'xxx' | Replace placeholder with target page |
| S10 | Unplanned downtime for the System must not exceed <xx hours/minutes> per | Incompl. | unfilled field '<xx hours/minutes>' | Fill threshold and period |
| S11 | … and/or remote commanding operations 10 consecutive times, the following actions shall | Incompl. | sentence ends mid-clause | Recover the missing action list |
| S12 | Should provide at least a basic level of job accounting, to aid in | Incompl. | ends at 'to aid in' boundary | Complete purpose; name the actor |
| S13 | The configuration window shall display the following non-modifiable information: | Incompl. | list introduction without list | Attach the referenced item list |
| S14 | Should one of these gates be lowered across an otherwise open entrance, the results could be catastrophic. | Non-test. | no verifiable behavior; hazard prose | Rewrite as testable interlock requirement |
| S15 | Sorting results should take less than 0.1 seconds. | Clean | measurable threshold (0.1 s) | Accept; verify with performance test |
| S16 | The system should be available 24 hours a day, 7 days a week. | Clean | explicit availability window | Accept; add uptime tolerance if needed |
| S17 | The DUAP System shall allow a system administrator to add, modify and delete user access rights. | Clean | observable admin actions; flat rule over-flags | Accept; no rewrite needed |
| S18 | A transaction rate of 2 CMIP transactions (sustained) per second shall be supported by each … | Clean | quantified sustained rate | Accept; verify in load test |
| S19 | The system shall generate an access log for every add, delete and edit action in the system. | Clean | enumerated auditable events | Accept; verify via log inspection |
| S20 | 3.1.5.1.3 Processing The THEMAS system shall maintain the ON/OFF status of each heating and cooling unit. | Clean | observable state maintenance | Accept; no rewrite needed |

## IX. DISCUSSION

The results support the value of explanation-oriented screening. The improvement does not come only from adding more rules; it comes from using a review structure that mirrors how engineers inspect requirements. A reviewer needs the defect type, the phrase or boundary that triggered the decision, and a rationale that can be accepted, rejected, or corrected.

The strongest practical finding is that extraction quality and validation quality cannot be separated. Many hard cases involved truncated list introductions, sentence fragments, placeholders, or merged formatting artifacts. These cases are not pure semantic defects, but they still affect the review process because they prevent a requirement from being validated as written.

The second finding is that ambiguity remains highly actionable review work. Vague capability terms need context before they can become testable acceptance criteria. The explainable workflow helps by isolating the relevant span rather than forcing reviewers to inspect an entire document.

The third finding concerns appropriate reliance [10], [11]. The workflow should be used for triage and review support, not autonomous acceptance or rejection of requirements. The human reviewer remains responsible for final judgment, especially when a vague term is acceptable because it is defined elsewhere, or when a requirement references an external standard.

## X. THREATS TO VALIDITY AND LIMITATIONS

*Single-reviewer labeling.* The 110-requirement gold set was adjudicated by a single reviewer, so subjectivity and annotation bias are possible, and borderline decisions between ambiguity and non-testability are particularly sensitive to individual judgment. A stronger design would use multiple independent annotators, report inter-rater agreement, and adjudicate

disagreements with domain experts; we see this as the most important next step.

*Gold-set size and sampling.* The evaluation covers 110 manually reviewed requirements curated from 9,165 extracted statements. The subset was designed for defect diversity rather than corpus-frequency representativeness, so the reported metrics describe screening behavior on curated review material, not expected performance over the full extraction.

*Class imbalance.* Non-testability has a support of one after review, because several machine-seeded non-testability rows were relabeled during adjudication. Per-class metrics for that label are therefore unstable and may under-represent this defect class; future sampling should deliberately target it.

*Within-corpus evaluation and extraction noise.* The comparison was performed on a curated subset of the same corpus used to develop the screens, so the results should not be interpreted as cross-domain generalization. Legacy formats, line wraps, tables, and list structures also caused fragmentation, and some labeled defects reflect extraction artifacts rather than authoring defects.

*Scope of the claims.* The study evaluates screening behavior and the explanation artifacts returned to reviewers; it does not yet evaluate deployment with professional requirements engineering teams, and it does not establish that a particular commercial or open-source LLM generalizes across requirements engineering tasks. Studies with practicing engineers, including how explanations affect reviewer trust and correction behavior, are future work.

## XI. Conclusion

This paper presented a human-centered workflow for explainable requirement defect screening across heterogeneous SRS documents. The workflow extracted 9,165 requirement-like statements from 79 documents, curated a manually reviewed gold set of 110 requirements, and compared three screening conditions on that gold set. The explainable precedence-based screen produced the strongest result, achieving 0.855 accuracy and 0.891 macro-F1.

The core conclusion is narrow but useful: explainable screening can improve practical requirements review when it provides a label, rationale, source span, and rewrite guidance, while preserving human authority over final decisions. The remaining work is to extend the reviewed gold set with multiple independent annotators and inter-rater agreement, to deliberately sample under-represented defect classes such as non-testability, and to evaluate the workflow with practicing requirements engineers in realistic review settings.

## References


[1] ISO/IEC/IEEE 29148:2018, *Systems and software engineering—Life cycle processes—Requirements engineering*, 2nd ed., ISO/IEC/IEEE, 2018.

[2] M. Glinz, "On non-functional requirements," in *Proc. 15th IEEE Int. Requirements Engineering Conf. (RE)*, 2007, pp. 21–26.

[3] L. Montgomery, D. Fucci, A. Bouraffa, L. Scholz, and W. Maalej, "Empirical research on requirements quality: a systematic mapping study," *Requirements Engineering*, vol. 27, no. 2, pp. 183–209, 2022.

[4] D. Zowghi and V. Gervasi, "On the interplay between consistency, completeness, and correctness in requirements evolution," *Information and Software Technology*, vol. 45, no. 14, pp. 993–1009, 2003.

[5] H. Femmer, D. Méndez Fernández, S. Wagner, and S. Eder, "Rapid quality assurance with requirements smells," *Journal of Systems and Software*, vol. 123, pp. 190–213, 2017.

[6] D. M. Berry and E. Kamsties, "Ambiguity in requirements specification," in *Perspectives on Software Requirements*, J. C. S. do Prado Leite and J. H. Doorn, Eds. Boston, MA, USA: Kluwer Academic Publishers, 2004, pp. 7–44.

[7] J. J. Norheim, E. Rebentisch, D. Xiao, L. Draeger, A. Kerbrat, and O. L. de Weck, "Challenges in applying large language models to requirements engineering tasks," *Design Science*, vol. 10, e16, 2024.

[8] Y. Uygun and V. Momodu, "Local large language models to simplify requirement engineering documents in the automotive industry," *Production & Manufacturing Research*, vol. 12, no. 1, 2024.

[9] D. Luitel, S. Hassani, and M. Sabetzadeh, "Improving requirements completeness: automated assistance through large language models," *Requirements Engineering*, vol. 29, no. 1, pp. 73–95, 2024.

[10] S. Amershi et al., "Guidelines for human-AI interaction," in *Proc. 2019 CHI Conf. Human Factors in Computing Systems*, 2019, pp. 1–13.

[11] B. Shneiderman, "Human-centered artificial intelligence: Reliable, safe & trustworthy," *Int. J. Human–Computer Interaction*, vol. 36, no. 6, pp. 495–504, 2020.

[12] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[13] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," arXiv:1702.08608, 2017.